

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 9

Name: S Svava Haasini

Roll No: 2520090170

Task 1: List Active Jobs

Question:

Implement how to list active jobs, display status information, retrieve job IDs, format output, and update state information.

Source Code:

```
GNU nano 7.2 T1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define JOBS 3

typedef struct
{
    int job_id;
    pid_t pid;
    char *state;
} Job;

int main(void)
{
    Job jobs[JOBS];

    printf("=== Active Job Listing ===\n");

    for (int i = 0; i < JOBS; i++)
    {
        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
            return 1;
        }

        if (pid == 0)
        {
            sleep(i + 2);
            exit(0);
        }

        jobs[i].job_id = i + 1;
    }
}
```

```

GNU nano 7.2 T1.c
    exit(0);
}

    jobs[i].job_id = i + 1;
    jobs[i].pid = pid;
    jobs[i].state = "Running";
}

printf("\nInitial Job List:\n");
printf("JOB ID\tPID\tSTATE\n");

for (int i = 0; i < JOBS; i++)
{
    printf("%d\t%d\t%s\n",
           jobs[i].job_id,
           jobs[i].pid,
           jobs[i].state);
}

for (int i = 0; i < JOBS; i++)
{
    waitpid(jobs[i].pid, NULL, 0);
    jobs[i].state = "Completed";
}

printf("\nUpdated Job List:\n");
printf("JOB ID\tPID\tSTATE\n");

for (int i = 0; i < JOBS; i++)
{
    printf("%d\t%d\t%s\n",
           jobs[i].job_id,
           jobs[i].pid,
           jobs[i].state);
}

printf("\nJob listing test completed.\n");

return 0;

```

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week9$ nano T1.c
haasini@SSH:~/OSSP_upload/Skill_Week9$ nano T1.c
haasini@SSH:~/OSSP_upload/Skill_Week9$ gcc -Wall -Wextra T1.c -o T1
haasini@SSH:~/OSSP_upload/Skill_Week9$ ./T1
=== Active Job Listing ===

Initial Job List:
JOB ID  PID    STATE
1       742    Running
2       743    Running
3       744    Running

Updated Job List:
JOB ID  PID    STATE
1       742    Completed
2       743    Completed
3       744    Completed

Job listing test completed.
haasini@SSH:~/OSSP_upload/Skill_Week9$ |

```

Observations:

1. A Job struct defines the job record structure containing job_id (job number), pid (process ID), and state (job status as string).
2. #define JOBS 3 sets the maximum number of concurrent jobs that can be tracked in the active job list.
3. fork() creates multiple child processes (3 in the example with PIDs 742, 743, 744) that execute independently while sleeping.
4. Each forked child receives job_id, pid, and state='Running' entries in the jobs array, establishing the initial job list.
5. The initial job list displays all active jobs with their assigned IDs, process IDs, and 'Running' status.
6. waitpid() monitors each job's completion status, and when a child completes, its state updates from 'Running' to 'Completed'.
7. The updated job list displays all completed jobs showing their final 'Completed' state and original PIDs.
8. Job listing test validates successful tracking of all job lifecycle stages: creation, execution, and completion states.

Task 2: Identify Target Job and Foreground Switching

Question:

Design how to identify target job, transfer terminal control, resume processes, wait for completion, and update job state.

Source Code:

```
GNU nano 7.2 T2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Job started. PID: %d\n", getpid());
        fflush(stdout);

        sleep(4);

        printf("Foreground job completed. PID: %d\n", getpid());
        fflush(stdout);

        exit(0);
    }

    printf("Background job created.\n");
    printf("Job PID: %d\n", pid);

    sleep(1);

    printf("\nSwitching job %d to foreground...\n", pid);
    printf("Terminal control assigned to foreground job.\n");

    waitpid(pid, NULL, 0);

    printf("Foreground job finished.\n");
    printf("Terminal control returned to parent.\n");

    return 0;
}
```

Output:

```
Job listing test completed.
haasini@SSH:~/OSSP_upload/Skill_Week9$ nano T2.c
haasini@SSH:~/OSSP_upload/Skill_Week9$ gcc -Wall -Wextra T2.c -o T2
haasini@SSH:~/OSSP_upload/Skill_Week9$ ./T2
Background job created.
Job PID: 801
Job started. PID: 801

Switching job 801 to foreground...
Terminal control assigned to foreground job.
Foreground job completed. PID: 801
Foreground job finished.
Terminal control returned to parent.
haasini@SSH:~/OSSP_upload/Skill_Week9$ |
```

Observations:

1. `fork()` creates a child process that will become the target job for a simplified foreground-switching demonstration.
2. The child process identifies itself as the foreground job (by printed message) and executes a `sleep()` operation simulating active work.
3. The parent process displays 'Background job created' and stores the job PID, then prints demonstration messages about process roles.
4. `waitpid(pid, NULL, 0)` is a blocking call that suspends the parent process while waiting for the specified child to complete execution.
5. The printed 'Switching job X to foreground' and 'Terminal control assigned to foreground job' are demonstration messages that illustrate the intended workflow, not indicators of actual terminal ownership manipulation (which would require `tcsetpgrp()`).
6. This implementation uses blocking `waitpid()` as a mechanism to ensure sequential execution and let the child process run independently while the parent waits.
7. When the child exits (after sleep completes), `waitpid()` returns and the parent resumes execution.
8. The printed message 'Foreground job finished' and 'Terminal control returned to parent' demonstrate the completion of the simplified foreground-switching workflow.

Task 3: Signal Delivery and Blocking Behavior

Question:

Analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior.

Source Code:

```
GNU nano 7.2 T3.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t received = 0;

void handler(int sig)
{
    if (sig == SIGUSR1)
        received = 1;
}

int main(void)
{
    struct sigaction sa;
    sigset_t block_set;

    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGUSR1, &sa, NULL) == -1)
    {
        perror("sigaction");
        return 1;
    }

    sigemptyset(&block_set);
    sigaddset(&block_set, SIGUSR1);

    if (sigprocmask(SIG_BLOCK, &block_set, NULL) == -1)
    {
        perror("sigprocmask");
        return 1;
    }

    printf("Process PID: %d\n", getpid());
    printf("SIGUSR1 is now blocked.\n");

    raise(SIGUSR1);

    printf("SIGUSR1 sent while blocked.\n");

    if (!received)
        printf("Signal handler has NOT executed yet.\n");

    printf("Unblocking SIGUSR1...\n");

    if (sigprocmask(SIG_UNBLOCK, &block_set, NULL) == -1)
    {
        perror("sigprocmask");
        return 1;
    }

    sleep(1);

    if (received)
        printf("SIGUSR1 delivered successfully after unblocking.\n");

    printf("Signal test completed.\n");

    return 0;
}
```

Output:

```

terminal control returned to parent.
haasini@SSH:~/OSSP_upload/Skill_Week9$ nano T3.c
haasini@SSH:~/OSSP_upload/Skill_Week9$ gcc -Wall -Wextra T3.c -o T3
haasini@SSH:~/OSSP_upload/Skill_Week9$ ./T3
Process PID: 909
SIGUSR1 is now blocked.
SIGUSR1 sent while blocked.
Signal handler has NOT executed yet.
Unblocking SIGUSR1...
SIGUSR1 delivered successfully after unblocking.
Signal test completed.
haasini@SSH:~/OSSP_upload/Skill_Week9$ |

```

Observations:

1. volatile sig_atomic_t received = 0 declares an atomic variable to safely track signal delivery across process contexts.
2. The handler() function is registered as the signal handler for SIGUSR1, executing only when the signal is delivered.
3. sigaction() registers the custom handler and sets up signal masking configuration for SIGUSR1 delivery.
4. sigemptyset() and sigaddset() configure which signals are blocked during handler execution, implementing signal masks.
5. sigprocmask(SIG_BLOCK, ...) blocks SIGUSR1 delivery, preventing the signal from reaching the process temporarily.
6. raise(SIGUSR1) sends SIGUSR1 to the process while it is blocked, queuing the signal for later delivery.
7. The message 'Signal handler has NOT executed yet' confirms that the blocked signal is pending but not yet delivered.
8. sigprocmask(SIG_UNBLOCK, ...) unblocks SIGUSR1, allowing the pending signal to be delivered immediately.
9. After unblocking, the handler executes and 'SIGUSR1 delivered successfully after unblocking' confirms signal delivery.
10. This demonstrates signal masking, blocking behavior, and the relationship between signal delivery and process signal masks.

Task 4: Resume Stopped Jobs and Background Job Recovery

Question:

Implement how to resume stopped jobs, send continue signals, update job status, maintain job table, and verify execution.

Source Code:

```
GNU nano 7.2 T4.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef struct
{
    int job_id;
    pid_t pid;
    char *state;
} Job;

int main(void)
{
    Job job;

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child job started. PID: %d\n", getpid());
        fflush(stdout);

        for (int i = 1; i <= 5; i++)
        {
            printf("Child working... step %d\n", i);
            fflush(stdout);
            sleep(1);
        }

        printf("Child job completed.\n");
        exit(0);
    }

    job.job_id = 1;
    job.pid = pid;
    job.state = "Running";

    sleep(1);

    printf("\nJob Table:\n");
    printf("JOB\tPID\tSTATE\n");
    printf("%d\t%d\t%s\n",
           job.job_id,
           job.pid,
           job.state);

    printf("\nStopping job %d using SIGSTOP...\n",
           job.job_id);

    kill(pid, SIGSTOP);
}
```



```

        job.job_id);

kill(pid, SIGSTOP);

waitpid(pid, NULL, WUNTRACED);

job.state = "Stopped";

printf("\nUpdated Job Table:\n");
printf("JOB\tPID\tSTATE\n");
printf("%d\t%d\t%s\n",
        job.job_id,
        job.pid,
        job.state);

sleep(2);

printf("\nSending SIGCONT to job %d...\n",
        job.job_id);

kill(pid, SIGCONT);

job.state = "Running";

printf("Job resumed successfully.\n");

printf("\nJob Table After Resume:\n");
printf("JOB\tPID\tSTATE\n");
printf("%d\t%d\t%s\n",
        job.job_id,
        job.pid,
        job.state);

waitpid(pid, NULL, 0);

job.state = "Completed";

printf("\nFinal Job Table:\n");
printf("JOB\tPID\tSTATE\n");
printf("%d\t%d\t%s\n",
        job.job_id,
        job.pid,
        job.state);

printf("\nBackground job recovery test completed.\n");

return 0;
}

```

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week9$ nano T4.c
haasini@SSH:~/OSSP_upload/Skill_Week9$ gcc -Wall -Wextra T4.c -o T4
haasini@SSH:~/OSSP_upload/Skill_Week9$ ./T4
Child job started. PID: 986
Child working... step 1

Job Table:
JOB      PID      STATE
Child working... step 2
1        986      Running

Stopping job 1 using SIGSTOP...

Updated Job Table:
JOB      PID      STATE
1        986      Stopped

Sending SIGCONT to job 1...
Job resumed successfully.

Job Table After Resume:
JOB      PID      STATE
1        986      Running
Child working... step 3
Child working... step 4
Child working... step 5
Child job completed.

Final Job Table:
JOB      PID      STATE
1        986      Completed

Background job recovery test completed.
haasini@SSH:~/OSSP_upload/Skill_Week9$ |

```

Observations:

1. A Job struct stores individual job metadata including job_id, pid, and state to track job lifecycle across state transitions.
2. fork() creates a child process (PID 986) that performs iterative work (steps 1-5) with sleep delays simulating background execution.
3. The initial job table shows the job in 'Running' state with its assigned PID and job ID.
4. kill(pid, SIGSTOP) sends SIGSTOP to the running job, causing it to suspend execution without terminating (state changes to 'Stopped').
5. The updated job table reflects the job's 'Stopped' state after SIGSTOP delivery, demonstrating state transition from Running to Stopped.
6. kill(pid, SIGCONT) sends SIGCONT to the stopped job, resuming execution from where it was suspended (state changes back to 'Running').
7. The 'Job resumed successfully' message confirms that the stopped process has resumed execution with SIGCONT.
8. Job table after resume shows the job in 'Running' state again, and the child completes its remaining work steps (3, 4, 5).
9. Final job table shows state transitions: Running → Stopped → Running → Completed, demonstrating complete job lifecycle management.
10. Background job recovery test validates that jobs can be suspended with SIGSTOP and resumed with SIGCONT while maintaining execution state and continuing work.